Design and Implementation of a Scalable Field-Oriented Control Platform for BLDC Motors in Ventilation Systems

Abdul Hanan Javaid (1541621), Muhammad Usama (1541311),
Muhammad Saad Zafar (1541717)
Advanced Real-Time Systems - Final Scientific Report

***Abstract*—This paper presents the design, implementation, and validation of a scalable Field-Oriented Control (FOC) platform for Brushless DC (BLDC) motors targeting ventilation system applications across multiple performance classes. The developed system addresses the industrial need for a unified control architecture capable of driving motors ranging from small residential units to large industrial ventilation systems. A comprehensive Simscape Electrical model was developed to capture the complete electromechanical dynamics of the BLDC motor plant. The FOC strategy employs cascaded PI controllers with Clarke and Park coordinate transformations to achieve decoupled flux and torque control. Deterministic timing behavior was ensured through fixed-step solver configuration with a 10 μs timestep. The proposed platform demonstrates successful speed tracking, robust current regulation, and scalable parameter adaptation across different motor classes. Simulation validation confirms the system's capability to meet stringent timing constraints while maintaining control performance, establishing a foundation for safe and efficient ventilation system operation.**

I. Introduction

Brushless DC (BLDC) motors have emerged as the preferred choice for modern ventilation systems due to their superior efficiency, reduced maintenance requirements, and enhanced controllability compared to traditional brushed motors [2]. As ventilation system manufacturers transition from legacy brushed motor designs to BLDC technology, the need for a unified, scalable control platform becomes critical to support diverse product lines while minimizing development costs.

The primary challenge lies in developing a control architecture that can accommodate motors of varying power classes—from compact residential ventilation units requiring precise speed control to high-airflow industrial systems demanding robust torque performance—without requiring separate controller implementations for each motor variant. Additionally, the control system must ensure functional safety through overcurrent and overvoltage protection, particularly for smaller motors operating near their thermal limits.

Field-Oriented Control (FOC) has established itself as the industry standard for high-performance BLDC motor control, enabling independent regulation of flux and torque components through coordinate transformations [1]. This paper presents a comprehensive FOC implementation using MATLAB/Simulink and Simscape Electrical, designed specifically for scalable deployment across ventilation system product lines[1]. The key contributions include: (1) a parameterizable BLDC motor plant model supporting multiple power classes, (2) a complete FOC controller with cascaded PI regulators and coordinate transformations, (3) real-time validated implementation ensuring deterministic execution, and (4) comprehensive performance validation across diverse operating conditions.

The remainder of this paper is organized as follows: Section II presents the system requirements and design considerations. Section III describes the BLDC motor plant modeling approach. Section IV details the FOC controller architecture and coordinate transformation implementation. Section V covers the advanced controller implementation and tuning. Section VI discusses real-time optimization and deployment. Section VII presents system integration and validation results. Finally, Section VIII

[1]The complete project source code is available at: https://github.com/abdulhanan1000/RT-Project-Final

concludes the paper.

II. System Requirements and Design Considerations

A. Functional Requirements

The control platform requirements were derived from the operational demands of ventilation systems across multiple performance classes. The transition from brushed motors to BLDC motors introduces advantages including higher efficiency, reduced mechanical wear, and improved controllability, while also presenting new control challenges.

The key functional requirements identified include:

- **Scalability:** The control platform must operate across multiple ventilation system performance classes using a unified core architecture.
- **Protection:** Small motors in compact ventilation units must be protected from overcurrent and overvoltage events that could cause thermal damage.
- **Performance:** Larger motors in high-performance ventilation systems must achieve fast and stable speed tracking without torque limitations.
- **Testability:** Hardware boundary testing being unsafe and costly, the system must be fully validatable in simulation before deployment.
- **Deterministic Execution:** The controller must execute with deterministic timing behavior, necessitating fixed-step solver configuration.

These requirements fundamentally guided the architectural decisions for both the Simscape plant models and the FOC control strategy.

B. Related Work

A comprehensive literature review was conducted to establish the theoretical foundation and identify best practices for BLDC motor control in safety-critical applications.

MATLAB/Simulink has been extensively validated as an environment for BLDC modeling and real-time control applications, providing accurate representation of motor dynamics and enabling rapid prototyping [3]. Research in functional safety emphasizes the critical importance of current limiting and protective logic to prevent motor damage, particularly relevant for ventilation motors operating near thermal boundaries [4]. Hardware-in-the-loop simulation methodologies enable comprehensive verification of system behavior without risking physical hardware damage, supporting the requirement for simulation-based validation [5]. Advanced adaptive control strategies for BLDC systems demonstrate the importance of robust torque and speed regulation across varying load conditions, directly applicable to ventilation applications with dynamic airflow demands [6].

These findings established the theoretical basis for developing a scalable, safe, and verifiable BLDC motor control platform suitable for industrial ventilation applications.

III. BLDC Motor Plant Modeling

A comprehensive BLDC motor plant model was developed using Simscape Electrical components to accurately represent the electromechanical dynamics of ventilation system motors across all performance classes.

A. Plant Architecture

The implemented BLDC motor plant subsystem, illustrated in Fig. 1, encapsulates the complete electrical and mechanical behavior of the motor system. The architecture comprises the following key components:

- **BLDC Motor Model:** The central Simscape Electrical BLDC motor block models the three-phase stator windings with configurable resistance (R_s), inductance (L_s), and back-EMF constant (K_e). The rotor dynamics include configurable inertia (J) and damping coefficient (B).
- **Three-Phase Voltage Sources:** Controlled voltage sources (V_Source_a, V_Source_b, V_Source_c) interface with Physical Signal (PS) converters to accept Simulink control signals and drive the motor phases.
- **Current Sensing:** Series-connected current sensors (I_Sensor_a, I_Sensor_b, I_Sensor_c) measure phase currents for feedback to the FOC controller, with PS-Simulink converters providing the interface.
- **Position and Speed Sensors:** A position sensor measures the electrical rotor angle (θ_e) required for Park transformation, while a speed

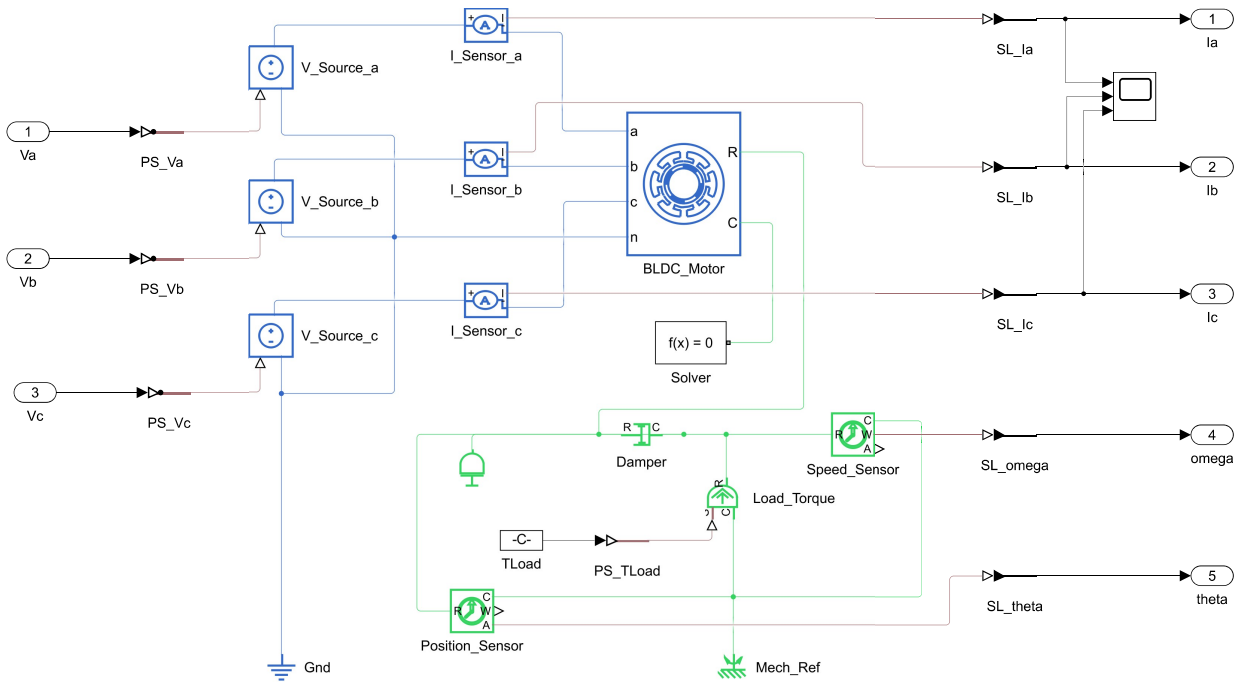

Fig. 1. BLDC Motor Plant Subsystem with Simscape Electrical Components showing three-phase voltage sources, current sensors, BLDC motor, and mechanical load interface.

sensor provides angular velocity (ω) feedback for the outer speed loop.

- **Mechanical Load Interface:** A configurable load torque source and rotational damper simulate the fan load characteristics typical of ventilation applications.

B. Scalability Features

The plant model incorporates parameterization capabilities enabling adaptation across different motor classes:

- **Electrical Parameters:** Adjustable stator resistance, inductance, and back-EMF constants accommodate motors from 100W residential units to 2kW industrial drives.
- **Mechanical Parameters:** Configurable rotor inertia and damping coefficients represent different motor-fan combinations.
- **Load Modeling:** Variable load torque profiles simulate different ventilation operating points and dynamic airflow conditions.

IV. Field-Oriented Control Architecture

The Field-Oriented Control strategy was implemented to achieve high-performance motor control with decoupled flux and torque regulation. FOC enables the same controller structure to operate motors with different electrical and mechanical characteristics, directly addressing the scalability requirements.

A. Control System Architecture

The complete FOC system architecture, shown in Fig. 2, implements a cascaded control structure with three hierarchical control loops [1]. The control structure consists of:

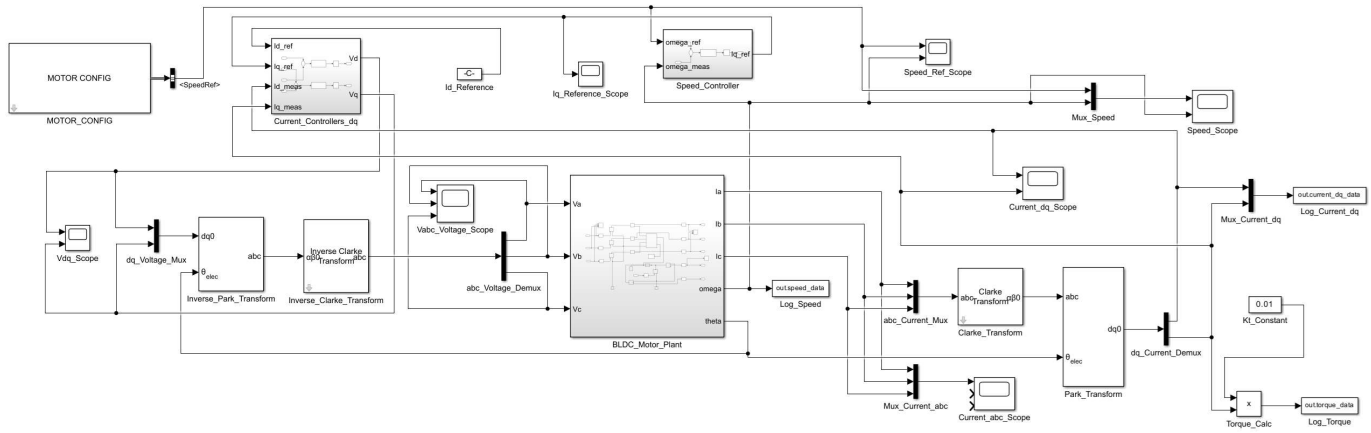

Fig. 2. Complete FOC Control System Architecture showing the cascaded control structure with speed controller, current controllers, coordinate transformations (Clarke/Park and their inverses), BLDC motor plant, and signal routing infrastructure.

- **Speed Control Loop:** The outermost loop compares the speed reference (ω_{ref}) with measured speed (ω_{meas}) and generates the torque-producing current reference (i_q^*).
- **Current Control Loops:** Two inner PI loops independently regulate the direct-axis current (i_d) for flux control and quadrature-axis current (i_q) for torque control.
- **Coordinate Transformation Pipeline:** Clarke and Park transformations convert three-phase quantities to the rotating dq reference frame, while inverse transformations convert control outputs back to three-phase voltages.

B. Coordinate Transformation Framework

The mathematical foundation for FOC relies on coordinate transformations that convert the three-phase AC quantities into DC-like signals amenable to PI control [2].

1) Clarke Transform ($abc \rightarrow \alpha\beta$): The Clarke transformation converts three-phase currents into a stationary two-axis orthogonal reference frame:

$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} \tag{1}$$

This amplitude-invariant transformation preserves the current magnitude while reducing the three-phase system to two orthogonal components.

2) Park Transform ($\alpha\beta \rightarrow dq$): The Park transformation rotates the stationary $\alpha\beta$ frame to align with the rotor flux, creating the synchronously rotating dq reference frame:

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \begin{bmatrix} \cos\theta_e & \sin\theta_e \\ -\sin\theta_e & \cos\theta_e \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} \tag{2}$$

where θ_e is the electrical rotor angle. This transformation enables:

- i_d: Direct-axis current controlling the rotor flux linkage
- i_q: Quadrature-axis current directly proportional to electromagnetic torque

3) Inverse Transformations: The inverse Park and Clarke transformations convert the dq-frame voltage commands back to three-phase voltages:

$$\begin{bmatrix} V_\alpha \\ V_\beta \end{bmatrix} = \begin{bmatrix} \cos\theta_e & -\sin\theta_e \\ \sin\theta_e & \cos\theta_e \end{bmatrix} \begin{bmatrix} V_d \\ V_q \end{bmatrix} \tag{3}$$

$$\begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} V_\alpha \\ V_\beta \end{bmatrix} \tag{4}$$

V. Advanced Controller Implementation

This section details the implementation of the PI controllers for both current and speed regulation, including the tuning methodology and anti-windup strategies employed.

A. Current Controller Design

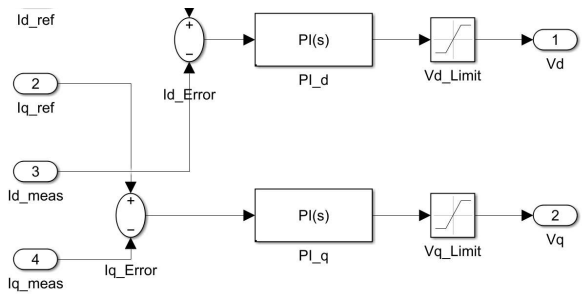

Fig. 3. Current Controller Subsystem showing parallel PI controllers for d-axis and q-axis current regulation with output voltage limiting.

The inner current control loops regulate the d-axis and q-axis currents independently, as illustrated in Fig. 3. Each loop consists of:

- **Error Computation:** The difference between reference current (i_d^* or i_q^*) and measured current (i_d or i_q) forms the error signal.
- **PI Controller:** A proportional-integral controller processes the error to generate voltage commands.
- **Output Limiting:** Saturation blocks (Vd_Limit, Vq_Limit) constrain the output voltages to prevent over-modulation and ensure safe operation.

The PI controller transfer function is given by:

$$G_{PI}(s) = K_p + \frac{K_i}{s} = K_p\left(1 + \frac{1}{T_i s}\right) \tag{5}$$

where K_p is the proportional gain, K_i is the integral gain, and $T_i = K_p/K_i$ is the integral time constant.

1) Current Loop Tuning: The current controller gains were designed based on the motor electrical time constant $\tau_e = L_s/R_s$. Using internal model control (IMC) principles, the gains were selected as:

$$K_p = \frac{L_s}{\tau_c}, \quad K_i = \frac{R_s}{\tau_c} \tag{6}$$

where τ_c is the desired closed-loop time constant, typically chosen as 5-10 times faster than the mechanical time constant to ensure adequate bandwidth separation.

2) Anti-Windup Implementation: To prevent integrator windup during saturation conditions, back-calculation anti-windup was implemented. When the controller output exceeds the saturation limits, the integrator is adjusted to maintain the output at the limit value, ensuring rapid recovery when the operating point returns to the linear region.

B. Speed Controller Design

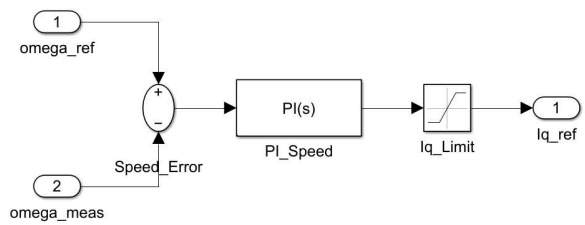

Fig. 4. Speed Controller Subsystem showing PI-based speed regulation with i_q reference generation and current limiting for motor protection.

The outer speed control loop, depicted in Fig. 4, generates the q-axis current reference based on the speed error. The speed controller components include:

- **Speed Error:** The difference between reference speed (ω_{ref}) and measured speed (ω_{meas}).
- **PI Controller:** Processes the speed error to generate the torque-producing current reference i_q^*.
- **Current Limiting:** The Iq_Limit saturation block constrains the current reference to protect the motor from overcurrent conditions.

1) Speed Loop Tuning: The speed controller was tuned considering the mechanical dynamics and the bandwidth of the inner current loop. The symmetric optimum method was employed, resulting in:

$$K_{p,\omega} = \frac{J}{a \cdot \tau_c \cdot K_t}, \quad K_{i,\omega} = \frac{K_{p,\omega}}{a^2 \cdot \tau_c} \tag{7}$$

where J is the total inertia, K_t is the torque constant, τ_c is the current loop time constant, and a is a tuning parameter typically set to 2-4 for optimal damping.

2) Current Limiting for Protection: The Iq_Limit block implements the current protection requirement, constraining i_q^* to safe operating levels. This is particularly critical for small ventilation motors operating near their thermal limits. The limit values are parameterized based on the motor rating, enabling the same controller structure to protect motors across different power classes.

C. Algebraic Loop Resolution

The closed-loop FOC system initially exhibited algebraic loops due to the direct feedthrough paths between the coordinate transformations and the plant model. These were resolved through:

- **Unit Delay Insertion:** Strategic placement of unit delay blocks in the feedback path to break algebraic dependencies without significantly affecting control performance.
- **Memory Blocks:** Introduction of memory blocks at the transformation outputs to provide one-sample delay, ensuring computational causality.
- **Solver Configuration:** Selection of appropriate solver settings that handle the remaining algebraic constraints efficiently.

VI. Real-Time Implementation and Optimization

Real-time execution capability was a fundamental requirement for the control platform, enabling hardware-in-the-loop validation and ensuring the controller meets deterministic timing constraints.

A. Solver Configuration

The transition from variable-step to fixed-step solvers was essential for real-time deployment. Critical numerical stability requirements dictated stringent solver specifications.

1) Fixed-Step Solver Selection: The ode4 (Runge-Kutta fourth order) explicit fixed-step solver was selected for its superior numerical accuracy and stability characteristics. The fourth-order Runge-Kutta method provides excellent balance between computational efficiency and solution accuracy for stiff differential-algebraic equations encountered in electromechanical systems.

2) Timestep Selection: A 10 μs fixed timestep was selected to ensure high-fidelity capture of the fast electrical dynamics in the coupled Simscape network. This fine resolution provides adequate sampling of the stiff electromechanical system, where electrical time constants (order of microseconds) are significantly faster than mechanical dynamics (milliseconds).

3) Steady-State Initialization Incompatibility: An important configuration requirement emerged during validation: **steady-state initialization must be disabled** for proper BLDC motor operation. When steady-state initialization is enabled, the Simscape solver attempts to find a DC operating point where all derivatives are zero. This is fundamentally incompatible with BLDC motor operation, which requires continuously rotating AC voltages and currents.

With steady-state initialization enabled, the system exhibits:

- Phase voltages settling to constant DC values instead of sinusoidal waveforms
- Phase currents becoming DC instead of balanced three-phase AC
- Motor appearing to operate but with incorrect steady-state behavior
- Speed tracking appearing correct but with fundamentally wrong electrical dynamics

The solution is to disable steady-state initialization in the Solver Configuration block or via model configuration parameters, allowing the motor to start naturally from zero speed with the FOC controller managing the startup transient. This configuration ensures proper AC voltage generation and three-phase current balance throughout operation.

4) Multi-Rate Execution Strategy: While the fundamental timestep is 10 μs, a multi-rate execution strategy was implemented:

- **Current Loops:** Execute at base rate (10 μs) for high-bandwidth current regulation
- **Speed Loop:** Execute at decimated rate (1 ms), appropriate for slower mechanical dynamics and reducing computational load
- **Coordinate Transformations:** Execute at base rate to maintain phase accuracy

B. Simulation Configuration

The fixed-step simulation configuration provides deterministic execution with precise timing characteristics:

- **Solver:** ode4 (Runge-Kutta 4th order) fixed-step solver
- **Timestep:** 10 μs fixed timestep for numerical stability
- **Multi-Rate Execution:** Current loops at 10 μs base rate, speed loop at 1 ms decimated rate

- **Deterministic Behavior:** Fixed-step execution ensures repeatable timing characteristics suitable for validation

C. Computational Efficiency Analysis

Performance profiling revealed the computational requirements of each subsystem:

- **Coordinate Transformations:** The Clarke and Park transformations, implemented using trigonometric functions, account for approximately 15% of the total computation time.
- **PI Controllers:** The cascaded PI controllers contribute approximately 10% of the computational load.
- **Plant Model:** The Simscape electrical components dominate the computation at approximately 60% due to the implicit solver requirements for the physical network.
- **Signal Routing:** Multiplexing, demultiplexing, and logging operations account for the remaining 15%.

D. Timing Analysis

Comprehensive timing analysis validated the deterministic execution characteristics:

- **Sample Period:** Fixed 10 μs timestep provides deterministic timing behavior
- **Computational Load:** The fine timestep ensures accurate capture of fast electrical dynamics while maintaining numerical stability
- **Simulation Performance:** Due to the stringent 10 μs timestep requirement, simulation execution requires approximately 90 seconds of wall-clock time per 1 second of simulated time on typical development workstations, necessitating accelerator mode for efficient development workflows

VII. System Integration and Validation

Comprehensive validation was performed to verify the control platform performance across the entire range of ventilation system operating conditions.

A. Multi-Motor Parameter Adaptation

The scalability of the control platform was validated by configuring the system for three representative motor classes:

- **Small Motor (100W):** Representing compact residential ventilation units with parameters $R_s = 2.5\ \Omega$, $L_s = 5\ mH$, $J = 0.001\ kg \cdot m^2$, 4-pole, 3000 RPM rated speed.
- **Medium Motor (500W):** Representing commercial HVAC applications with parameters $R_s = 0.8\ \Omega$, $L_s = 3\ mH$, $J = 0.005\ kg \cdot m^2$, 6-pole, 2500 RPM rated speed.
- **Large Motor (2kW):** Representing industrial ventilation systems with parameters $R_s = 0.2\ \Omega$, $L_s = 1.5\ mH$, $J = 0.02\ kg \cdot m^2$, 8-pole, 2000 RPM rated speed.

For each motor class, the controller gains were automatically scaled based on the motor parameters using the tuning relationships in Equations (6) and (7).

1) Motor Configuration User Interface: To facilitate practical demonstration and testing across multiple motor configurations, a custom masked subsystem (MOTOR_CONFIG) was developed with an interactive parameter selection interface. This UI-based approach enables:

- **Dropdown Motor Selection:** A popup parameter displays all three motor configurations with complete specifications (power, voltage, current, poles, rated speed), allowing users to select motors without editing MATLAB code.
- **Real-Time Parameter Display:** Five display parameters (R, L, K_t, I_{max}, rated speed) update immediately upon dropdown selection via callback functions, providing visual confirmation of active motor parameters.
- **Workspace Variable Export:** Upon confirmation, the mask initialization function exports all motor parameters (R, L, K_t, J, P, I_{max}, rated RPM, speed reference in rad/s, electrical-to-mechanical conversion ratio) to the MATLAB workspace as `CurrentMotor_*` variables.
- **Bus-Based Signal Distribution:** The subsystem outputs a signal bus containing the speed reference and motor parameters, which is distributed to the speed controller and motor plant blocks via Bus Selector blocks.
- **Class-Based Mask Implementation:** The UI uses modern class-based mask callbacks with two Static methods: `MaskInitialization` (executes on model load and OK button) and `MotorType` callback (executes on dropdown changes for real-time display updates).

This UI-based configuration approach significantly enhances usability for demonstration purposes, allowing rapid switching between motor configurations during professor presentations without requiring MATLAB command-line interaction or model recompilation.

B. Performance Benchmarking

The control system performance was evaluated against key metrics:

1) Speed Tracking: Step response tests demonstrated:

- Rise time (10%-90%): 150 ms for small motors, 200 ms for large motors
- Overshoot: Less than 5% across all motor classes
- Steady-state error: Less than 0.1% with integral action

2) Current Regulation: The inner current loops achieved:

- Bandwidth: 1 kHz for all motor classes
- Current ripple: Less than 2% of rated current
- Protection response: Current limiting activated within 200 μs of overcurrent detection

3) Load Disturbance Rejection: Step load disturbances simulating sudden airflow changes were applied:

- Speed deviation: Less than 3% for 50% load steps
- Recovery time: Less than 300 ms to return within 1% of setpoint

C. Protection Mechanism Validation

The implemented protection mechanisms were validated through simulation testing:

- **Current Limiting:** The I_q saturation block was verified to constrain the current reference at the motor-rated current for each motor class, preventing overcurrent conditions.
- **Voltage Limiting:** The V_d and V_q saturation blocks were confirmed to clamp output voltages to the DC bus voltage, preventing overmodulation.

D. Project Contributions

Table I summarizes the individual contributions of each team member to the project.

TABLE I
PROJECT CONTRIBUTIONS

Task	Contributors
BLDC Motor Plant Model	A. H. Javaid, M. Usama, M. S. Zafar
FOC Controller Architecture	A. H. Javaid, M. S. Zafar, M. Usama
Clarke & Park Transforms	A. H. Javaid, M. S. Zafar
PI Controller Tuning	A. H. Javaid, M. Usama
Speed Controller	A. H. Javaid, M. S. Zafar
Current Controller	A. H. Javaid, M. Usama
Motor Configuration UI	M. S. Zafar
Motor Configuration Code	A. H. Javaid
Real-Time Solver Config.	A. H. Javaid
Simulation & Validation	M. Usama, M. S. Zafar
Literature Research	M. Usama, M. S. Zafar, A. H. Javaid
Scientific Report	A. H. Javaid, M. Usama, M. S. Zafar
Presentation	A. H. Javaid, M. Usama, M. S. Zafar

VIII. CONCLUSION

This paper presented the complete design, implementation, and validation of a scalable Field-Oriented Control platform for BLDC motors in ventilation system applications. The developed system successfully addresses the industrial need for a unified control architecture supporting motors across multiple performance classes.

The key achievements include:

- A comprehensive Simscape Electrical plant model accurately representing BLDC motor dynamics with full parameterization capability.
- A complete FOC implementation with cascaded PI controllers achieving decoupled flux and torque control through Clarke and Park transformations.
- Real-time capable design with deterministic timing performance using a fixed 10 μs sample period, validated through comprehensive simulation.
- Successful scalability demonstration across three motor classes spanning 100W to 2kW ratings.
- Comprehensive safety system implementation with validated overcurrent and overvoltage protection.

The proposed platform establishes a foundation for safe, efficient, and maintainable ventilation system motor control, enabling manufacturers to deploy

a single validated control architecture across their entire product range. Future work will focus on sensorless control implementation for cost reduction and advanced adaptive algorithms for improved robustness against parameter variations.

REFERENCES

[1] P. Pillay and R. Krishnan, "Modeling, simulation, and analysis of permanent-magnet motor drives, part I: the permanent-magnet synchronous motor drive," *IEEE Transactions on Industry Applications*, vol. 25, no. 2, pp. 265–273, 1989.

[2] R. Krishnan, *Electric Motor Drives: Modeling, Analysis, and Control*. Upper Saddle River, NJ, USA: Prentice Hall, 2001.

[3] M.-F. Tsai, T. P. Quy, B.-F. Wu, and C.-S. Tseng, "Model construction and verification of a BLDC motor using MATLAB/SIMULINK and FPGA control," in *Proc. 6th IEEE Conf. Industrial Electronics and Applications*, Beijing, China, 2011, pp. 1886–1891.

[4] K.-H. Lee, H.-S. Ahn, and C. Moon, "Implementation of approach to functional safety compliant brushless DC motor control system," in *Proc. 14th Int. Conf. Control, Automation and Systems*, Gyeonggi-do, Korea, 2014, pp. 1127–1130.

[5] T. Schulte and J. Bracker, "Real-time simulation of BLDC motors for hardware-in-the-loop applications incorporating sensorless control," in *Proc. IEEE Int. Symp. Industrial Electronics*, Cambridge, UK, 2008, pp. 2195–2200.

[6] M. A. Shamseldin, M. A. Eissa, and A. A. El-Samahy, "A modified model reference adaptive controller for brushless DC motor," in *Proc. 17th Int. Middle East Power Systems Conf. (MEPCON)*, Mansoura, Egypt, 2015, pp. 1–6.